REDEMPTION CITY NAVIGATOR

Architecture & System Design Document

Offline-first navigation, local discovery, chalets, mobility, and marketplace support for Redemption City.

Submission Version: MVP Prototype | Prepared: June 20, 2026

GitHub: https://github.com/Supreme5050/redemption-city-navigator

1. Executive Summary

Mission

Redemption City Navigator is a mobile-first navigation and local services MVP designed to reduce confusion, movement stress, and service discovery problems inside Redemption City. It combines offline-first place search, local route guidance, chalet discovery, Sunday Shuttle concepts, and a future multi-vendor Market Hub into one coordinated city-assistance platform.

The platform is built as an Expo React Native application with TypeScript and Expo Router. The MVP uses static local datasets for places, road corridors, chalets, vendors, and products so that core discovery can work even before a full backend is connected. This architecture is intentional: Redemption City has many local places and movement patterns that are not fully represented by external map providers, so the app must build and own a verified local knowledge layer.

SOLUTION NAME	Redemption City Navigator
PRIMARY CATEGORY	Safety, mobility, local navigation, and community services
PRIMARY USERS	Visitors, worshippers, residents, chalet guests, vendors, volunteers, and shuttle coordinators
PROTOTYPE TYPE	Expo React Native mobile MVP with offline-first local datasets
REPOSITORY	https://github.com/Supreme5050/redemption-city-navigator

2. Problem Context

- Visitors and first-time worshippers often struggle to locate roads, chalets, offices, churches, vendors, and key camp facilities without asking multiple people.
- Many internal roads and landmarks are not reliably available in external mapping platforms at the level of detail needed for camp movement.
- Sunday movement is uniquely painful because commercial transportation availability changes around worship schedules and large event flows.
- Local commerce is fragmented; customers need a safer way to discover vendors and buy services without direct trust risk.
- A mobile solution must remain useful under unstable connectivity, especially during high-traffic camp events.

3. Product Goals and Scope

GOAL 1	Provide offline-first search for important places, roads, chalets, vendors, and services.
GOAL 2	Provide local navigation guidance using verified coordinates rather than relying entirely on external map data.
GOAL 3	Create a foundation for Sunday Shuttle, Market Hub,

	chalet discovery, and event-day mobility support.
GOAL 4	Prepare the architecture for future backend services such as Supabase, vendor onboarding, escrow payment flows, and admin verification.
OUT OF CURRENT MVP SCOPE	Fully automated turn-by-turn navigation, live traffic prediction, production payments, full vendor KYC, and AR navigation are future phases.

4. High-Level System Architecture

```
Mobile App (Expo React Native)
  |
  |-- UI Layer: Expo Router screens, tabs, components
  |-- Local Data Layer: places.ts, verifiedRoutes.ts, chalets.ts, vendors.ts, products.ts
  |-- Service Layer: local route builder, search logic, future payment/order services
  |-- Device Capabilities: GPS via expo-location, maps via react-native-maps
  |
Future Backend Layer (Supabase / API)
  |-- Auth, vendor profiles, product inventory, order tracking, wallets, escrow states
  |-- Admin verification, dispute handling, delivery proof, notifications
```

Design Principle

The MVP is intentionally offline-first. Static datasets act as a local knowledge base while the product matures toward verified backend-managed data. This avoids blocking core navigation and discovery features on internet availability.

5. Technology Stack

FRONTEND	Expo React Native with TypeScript
NAVIGATION	Expo Router with tab-based and route-based screens
MAPS	react-native-maps with hybrid/satellite-capable rendering
LOCATION	expo-location for GPS detection
DATA TODAY	Local TypeScript data files for places, verified routes, chalets, vendors, products, and services
BACKEND LATER	Supabase for authentication, database, storage, order tracking, and admin workflows
REPOSITORY	Git + GitHub

6. Core Modules

Module	Purpose	Current MVP State	Future Extension
Home / Dashboard	Entry point for major services	Working navigation hub and	Personalized suggestions and

Module	Purpose	Current MVP State	Future Extension
	and quick discovery.	screen structure.	event-aware alerts.
Map & Search	Find places and route through saved coordinates.	Search works with local place data; route accuracy is being calibrated with verified road points.	Calibration-driven turn-by-turn route graph with road segments.
Places Database	Offline knowledge base of roads, churches, offices, food spots, and landmarks.	Static TypeScript records.	Admin-editable database with validation workflow.
Chalets	Help users discover accommodation options and directions.	Screen and data foundation present.	Availability, booking, pricing, and hospitality integration.
Market Hub	Local commerce and vendor discovery.	Demo/static marketplace structure.	Multi-vendor onboarding, wallets, escrow, delivery tracking, dispute flow.
Sunday Shuttle	Help solve Sunday movement constraints.	Concept and screen foundation.	Verified pickup points, seat offers, safety controls, and route matching.
AI Assistant	Guide users through search, locations, and local service questions.	Planned floating assistant interaction.	RAG-style local knowledge assistant with admin-curated content.
AR Navigation	Future visual guidance for internal routes.	Not active in current MVP.	Camera-based guidance after coordinate accuracy is stable.

7. Data Architecture

- places.ts stores searchable locations, names, aliases, categories, zones, address hints, and coordinates.
- verifiedRoutes.ts stores saved route corridors as ordered latitude/longitude road points.
- chalets.ts stores accommodation demo data for discovery and future booking integration.
- vendors.ts and products.ts provide the MVP foundation for Market Hub browsing.
- services/localRouteService.ts contains route-building utilities and distance calculations used by the map screen.

Example local data relationship

```
Place: Heritage Road
  -> category: Road / Location
  -> coordinates: latitude, longitude
  -> aliases: Heritage Rd, Lemoshe, Heritage
  -> route match: verifiedRoutes entry containing road/junction points
```

8. Map and Navigation Design

- The map screen uses a hybrid/satellite-capable map view for 3D-style inspection of roads, buildings, and landmarks.
- Search resolves a destination from local data, not from an external web API.

- The current route line is drawn from saved coordinates in verifiedRoutes.ts.
- GPS is used to show the user location, but it should not invent fake road lines where verified road data is missing.
- The route calibration workflow collects road bend and junction coordinates so routes can follow true internal roads over time.

Current Navigation Status

The MVP map, GPS, search, and route rendering are working. The remaining navigation work is route-coordinate accuracy: the road data must be calibrated using true junction and bend points for each key corridor.

9. Market Hub and Escrow Direction

The Market Hub is designed as a future multi-vendor marketplace, not a single-person shop. Vendors will eventually sign in, manage store profiles, list products/services, track orders, and see wallet/earnings status.

- Customers browse products across many vendors before choosing a vendor.
- Payments should go into the app/system first rather than directly to vendors.
- Funds are held until delivery is confirmed through buyer confirmation, OTP/PIN, delivery proof, GPS/timestamp evidence, or admin review.
- The model should include dispute reporting, auto-release rules, trust scores, and admin arbitration in later phases.

10. Security, Safety, and Privacy Considerations

- Location use should be permission-based and transparent to the user.
- Sensitive user/vendor/payment data should not be stored in the app bundle in production.
- Future backend access should use authenticated sessions, row-level security, and role-based permissions.
- Escrow and delivery confirmation should protect both buyers and vendors against abuse.
- Admin workflows should log verification, dispute, delivery, and payment release events.

11. MVP Status and Roadmap

Module	Purpose	Current MVP State	Future Extension
Current MVP	Expo mobile prototype with local data, app screens, search, maps, Market Hub foundation, chalets, and mobility concepts.	Built and pushed to GitHub.	Prepare demo video and submission assets.
Phase 1	Coordinate calibration for key roads and places.	In progress.	Add road-bend/junction coordinate capture and verified route review.
Phase 2	Backend connection and data management.	Planned.	Supabase auth, database, storage, admin data tools.
Phase 3	Market Hub production flow.	Planned.	Vendor onboarding, products, carts, escrow, delivery proof, disputes.
Phase 4	Mobility and AI	Planned.	Sunday Shuttle matching, AI

Module	Purpose	Current MVP State	Future Extension
	enhancements.		assistant, AR route overlay.

12. Risks and Mitigation

ROUTE ACCURACY RISK	Mitigation: use a calibration map to collect true road bend and junction coordinates before claiming full turn-by-turn accuracy.
DATA RELIABILITY RISK	Mitigation: introduce admin verification for places, roads, vendors, chalets, and shuttle points.
TRUST AND SAFETY RISK	Mitigation: use verified pickup points, delivery proof, escrow, admin dispute review, and role-based permissions.
CONNECTIVITY RISK	Mitigation: keep important location data offline-first and sync updates when internet is available.
SCALE RISK	Mitigation: move static data to Supabase tables with caching and local fallback.

13. Conclusion

Redemption City Navigator is a practical MVP for a real local movement and discovery problem. Its strongest architectural decision is the offline-first local knowledge base. The product already has a strong foundation for search, map display, service discovery, chalets, mobility, and marketplace expansion. The immediate technical priority is to complete route-coordinate calibration, then connect the app to a backend for verified dynamic data and production workflows.

END OF DOCUMENT